

TAos Programmer's Manual

This manual documents the functions and types found in the TAOS source files. It is intended for developers who wish to extend or maintain the system, providing clear descriptions of each function, parameter, return value, and custom type.

Kernel Functions

int serial_poll(device dev, char *buffer, size_t len)

Description:

Reads input from the serial device using polling. Handles special keys (arrows, delete, backspace).

Parameters:

- dev: Serial device identifier.
- buffer: Pointer to buffer to store input.
- len: Maximum number of bytes to read.

Return Value:

- Number of bytes read (excluding null terminator)

PCB Kernel Functions

pcb* pcb_allocate(void)

Description:

Allocates memory for a new PCB, including its stack, and performs reasonable initialization with default values (state = NEW, priority = 0, default stack size).

Parameters:

- None

Return Value:

- Pointer to the newly allocated PCB
- NULL if allocation fails

int pcb_free(pcb* p)

Description:

Frees the memory allocated for a PCB, including its stack, using sys_free_mem.

Parameters:

- p: Pointer to the PCB to free

Return Value:

- 0 on success
- -1 if input is invalid

pcb* pcb_setup(const char *name, int processor_class, int priority)**Description:**

Allocates a new PCB, initializes it with the given process name, processor class, and priority, and sets its state to *Ready* and *Not-Suspended*.

Parameters:

- name: Name of the process
- processor_class: Process class (0 = System, 1 = User)
- priority: Process priority (0 = highest, 9 = lowest)

Return Value:

- Pointer to the newly created PCB
- NULL if parameters are invalid or allocation fails

pcb* pcb_find(const char* name)**Description:**

Searches all process queues (Ready, Blocked, Suspended) for a process with the given name.

Parameters:

- name: Process name to search for

Return Value:

- Pointer to the PCB if found
- NULL if not found

static pcb* search_queue(queue *q, const char *name)**Description:**

Helper function used internally by `pcb_find` to search for a PCB with the given name in a specific queue. Iterates through the queue's linked list until a match is found.

Parameters:

- q: Queue to search through
- name: Process name to search for

Return Value:

- Pointer to the PCB if found
- NULL if not found

Notes:

- This function is marked static and is intended for internal use only.
- See also: `pcb_find`

void pcb_insert(pcb* p)

Description:

Inserts a PCB into the appropriate queue based on its state and priority.

- Ready processes are inserted in priority order (0 = highest, 9 = lowest).
- Blocked and Suspended processes are inserted at the tail of their queues.

Parameters:

- `p`: Pointer to the PCB to insert

Return Value:

- None

int pcb_remove(pcb* p)

Description:

Removes a PCB from its current queue without freeing its memory.

Parameters:

- `p`: Pointer to the PCB to remove

Return Value:

- 0 on success
- -1 if input is invalid
- -2 if PCB not found in any queue

Memory Manager Functions

void initialize_heap(size_t heap_size)

Description:

Initializes the heap memory manager by allocating a single large free block using `kmalloc()`.

Creates an MCB for this block, places it on the free list, and sets the allocated list to empty.

Parameters:

heap_size: Total heap size in bytes (does not include MCB size)

Return Value:

None

void allocate_memory(size_t size)**Description:**

Allocates memory from the heap using a first-fit algorithm. Splits free blocks when needed and updates MCBs. The allocated block is added to the allocated list.

Parameters:

size: Number of bytes requested

Return Value:

Pointer to the start of the allocated block (not the MCB)

NULL on error

int free_memory(void ptr)**Description:**

Frees a previously allocated memory block and places it on the free list. Adjacent free blocks are merged to reduce fragmentation.

Parameters:

ptr: Pointer to the start of an allocated block (not the MCB address)

Return Value:

0 on success

Non-zero on error

User Command Functions

void version(void)

Description:

Prints the current version of TAOS to the output.

Parameters:

None.

Return Value:

None.

void help(void)

Description:

Displays a list of available commands and their brief descriptions.

Parameters:

None.

Return Value:

None.

void help_with(char* command_name)

Description:

Displays detailed help for a specific command.

Parameters:

- command_name: Name of the command (string).

Valid Values:

Any valid command name (exit, version, getdate, setdate, gettime, settime).

Return Value:

None.

void get_date(void)

Description:

Reads the current date from the RTC and prints it in MM/DD/YYYY format.

Parameters:

None.

Return Value:

None.

void set_date(char *date_string)**Description:**

Sets the system date using a string in MM/DD/YYYY format. Leap year and specific months are considered.

Parameters:

- date_string: Date string (e.g., "09/12/2025").

Valid Values:

- Month: 1-12
- Day: 1-31 (Months with less than 31 days are included)
- Year: 0-9999

Return Value:

None. Prints error messages for invalid input.

void get_time(void)**Description:**

Reads the current time from the RTC and prints it in HH:MM:SS format.

Parameters:

None.

Return Value:

None.

void set_time(char *time_string)

Description:

Sets the system time using a string in HH:MM:SS format.

Parameters:

- time_string: Time string (e.g., "11:00:24").

Valid Values:

- Hour: 0-23
- Minute: 0-59

- Second: 0-59

Return Value:

None. Prints error messages for invalid input.

int shutdown(void)**Description:**

Prompts the user for confirmation before shutting down the system.

Parameters:

None.

Return Value:

- 1 if user confirms shutdown
- 0 if cancelled
- -1 for invalid response

PCB User Functions

int create_pcb(const char *name, int processor_class, int priority)**Description:**

Creates a new PCB with the specified name, processor class, and priority. The new PCB is initialized, inserted into the appropriate queue, and marked as *Ready, Not-Suspended*.

Parameters:

- name: Name of the PCB (string, must be unique)
- processor_class: 0 = System, 1 = User
- priority: Priority of the process (0 = highest, 9 = lowest)

Return Value:

- 0 on success
- -1 on invalid parameters or duplicate name

int delete_pcb(const char *name)**Description:**

Deletes a PCB with the specified name. The PCB is removed from its queue, memory is freed, and the process is destroyed.

Parameters:

- name: Name of the PCB to delete

Return Value:

- 0 on success
- -1 if the PCB does not exist, is invalid, or is a System process

int block_pcb(const char* name)**Description:**

Moves a PCB into the *Blocked* state and inserts it into the blocked queue.

Parameters:

- name: Name of the PCB to block

Return Value:

- 0 on success
- -1 if the PCB does not exist or is already blocked

int unblock_pcb(const char* name)**Description:**

Moves a PCB into the *Ready* state and inserts it into the ready queue.

Parameters:

- name: Name of the PCB to unblock

Return Value:

- 0 on success
- -1 if the PCB does not exist or is already ready

void show_pcb(const char *name)**Description:**

Displays detailed information about the specified PCB, including:

- Name
- Class (System/User)
- State (New, Ready, Blocked)
- Suspension status
- Priority

Parameters:

- name: Name of the PCB to display

Return Value:

- None (prints output to console)

int set_pcb_priority(const char* name, int new_priority)

Description:

Updates the priority of the specified PCB. The PCB is removed from its current queue and reinserted into the appropriate queue with the new priority.

Parameters:

- name: Name of the PCB to update
- new_priority: New priority value (0–9)

Return Value:

- 0 on success
- Negative values on error (-1 invalid name, -2 invalid priority, -3 PCB not found, -4 remove failed)

int suspend_pcb(const char* name)

Description:

Updates the priority of the specified PCB. The PCB is removed from its current queue and reinserted into the appropriate queue with the new priority.

Parameters:

- name: Name of the PCB to update
- new_priority: New priority value (0–9)

Return Value:

- 0 on success
- Negative values on error (-1 invalid name, -2 invalid priority, -3 PCB not found, -4 remove failed)

int suspend_pcb(const char* name)

Description:

Suspends the specified PCB by setting its suspension flag and moving it into the suspended queue.

Parameters:

- name: Name of the PCB to suspend

Return Value:

- 0 if already suspended
- Negative values on error (-1 invalid name, -2 PCB not found, -3 system process, -4 remove failed)

int resume_pcb(const char* name)

Description:

Resumes the specified PCB by clearing its suspension flag and moving it back into the appropriate queue.

Parameters:

- name: Name of the PCB to resume

Return Value:

- 0 if already not suspended
- Negative values on error (-1 invalid name, -2 PCB not found, -4 remove failed)

void show_ready_pcb(void)**Description:**

Displays all PCBs currently in the *Ready* queue. For each process, prints:

- Name
- Class (System/User)
- State
- Suspension status
- Priority

Parameters:

- None

Return Value:

- None (prints output to console)

void show_blocked_pcb(void)**Description:**

Displays all PCBs currently in the *Blocked* queue. For each process, prints:

- Name
- Class (System/User)
- State
- Suspension status
- Priority

Parameters:

- None

Return Value:

- None (prints output to console)

void show_all_pcb(void)**Description:**

Displays all PCBs across the *Ready*, *Blocked*, and *Suspended* queues. For each process, prints:

- Name
- Class (System/User)
- State
- Suspension status
- Priority

Parameters:

- None

Return Value:

- None (prints output to console)

MCB User Command Functions

void usr_allocate_memory(const char args)**Description:**

Allocates heap memory using `allocate_memory()` and prints the hexadecimal address of the allocated block (not the MCB address). Prints an error message if allocation fails.

Parameters:

args: Size of the requested allocation (decimal)

Return Value:

None

void usr_free_memory(const char args)**Description:**

Frees heap memory using `free_memory()` and prints a success or error message.

Parameters:

args: Address of the memory block to free (hexadecimal, not the MCB address)

Return Value:

None

void usr_show_allocated(const char args)**Description:**

Displays all allocated memory blocks. Prints each block's start address (hexadecimal, not MCB address) and size (decimal).

Parameters:

args: Should be NULL or empty (command takes no arguments)

Return Value:

None

void usr_show_free(const char args)**Description:**

Displays all free memory blocks. Prints each block's start address (hexadecimal, not MCB address) and size (decimal).

Parameters:

args: Should be NULL or empty (command takes no arguments)

Return Value:

None

Utility Functions

int atoi(const char *s)**Description:**

Converts a string to an integer.

Parameters:

- s: Null-terminated string representing a number.

Return Value:

- Parsed integer value
- -1 if the string is not a valid number

char *itoa(int n, char *str)**Description:**

Converts an integer to a string.

Parameters:

- n: Integer to convert.
- str: Buffer to store the result (should be large enough).

Return Value:

- Pointer to the resulting string

unsigned int i_to_bcd(int num)**Description:**

Converts an integer to Binary-Coded Decimal (BCD) format.

Parameters:

- num: Integer to convert (non-negative).

Return Value:

- BCD representation of the integer
- 0xFFFFFFFF if input is negative

int bcd_to_i(unsigned int bcd)**Description:**

Converts a BCD value to an integer.

Parameters:

- bcd: BCD value.

Return Value:

- Integer representation
- -1 if BCD is invalid

void printf(const char *string, ...)

Description:

Formatted output function supporting %d, %c, %s, and %%. Supports zero-padded integers (e.g., %02d).

Parameters:

- string: Format string.
- ...: Arguments matching format specifiers.

Return Value:

None.

void kputc(char c)**Description:**

Outputs a single character to the system output.

Parameters:

- c: Character to output.

Return Value:

None.

void kputs(const char *c)**Description:**

Outputs a null-terminated string to the system output.

Parameters:

- c: String to output.

Return Value:

None.

Queue Utility Functions

queue* create_queue(void)**Description:**

Allocates and initializes a new queue structure. The queue is empty upon creation.

Parameters:

- None

Return Value:

- Pointer to the newly created queue
- NULL if allocation fails

void enqueue(queue* q, pcb* p)**Description:**

Inserts a PCB into the specified queue. The PCB is added to the tail of the queue.

Parameters:

- q: Queue to insert into
- p: Pointer to the PCB to enqueue

Return Value:

- None

Notes:

- If q or p is NULL, no operation is performed.

pcb* dequeue(queue* q)**Description:**

Removes and returns the PCB at the head of the queue.

Parameters:

- q: Queue to remove from

Return Value:

- Pointer to the dequeued PCB
- NULL if the queue is empty or invalid

int is_empty(queue* q)**Description:**

Checks whether the specified queue is empty.

Parameters:

- q: Queue to check

Return Value:

- 1 if the queue is empty
- 0 if the queue is not empty

- -1 if the queue is invalid (NULL)

Command Handler

void commhand(void)

Description:

Main command handler loop. Displays welcome message, prompts for input, and dispatches commands.

Parameters:

None.

Return Value:

None.

void execute_command(char *command)

Description:

Parses and executes a command string entered by the user.

Parameters:

- command: Command string (may include arguments).

Return Value:

None. Prints error messages for invalid commands or arguments.

Command History

void hist_add(char* command);

Description:

Adds a new command string to the history. Entries are ordered by insertion order. If the history is full, the oldest entry is overwritten.

Parameters:

- command: Command string to add to history

Return Value:

- None

char* hist_next(void):

Description:

Moves the index forward in the command history and returns the command at the new index. Does not wrap around if already at the newest entry.

Parameters:

- None

Return Value:

- Pointer to the next command string in history
- NULL if at the end of history

char* hist_previous(void):

Description:

Moves the index backward in the command history and returns the command at the new index. Does not wrap around if already at the oldest entry.

Parameters:

- None

Return Value:

- Pointer to the previous command string in history
- NULL if at the start of history

char* hist_get(int i)

Description:

Retrieves the command string stored at a specific index in the history.

Parameters:

- i: Index in the command history

Return Value:

- Pointer to the command string at index *i*
- NULL if index is invalid

void hist_set(int i, char* command)

Description:

Replaces the command at a specific index in history with a new command string.

Parameters:

- i: Index in the command history
- command: New command string to set at the index

Return Value:

- None

void hist_index_set(int i)**Description:**

Sets the current index pointer for navigating the command history.

Parameters:

- i: Index to set as the current position

Return Value:

- None

int hist_get_index(void)**Description:**

Retrieves the current index pointer used in command history navigation.

Parameters:

- None

Return Value:

- Current index value

Notes for Extending TAOS

- All I/O is performed via sys_req and serial port functions.
- Command parsing is handled in execute_command.

- Add new commands by extending `execute_command` and implementing corresponding functions.
- Use `printf` for formatted output; it supports basic format specifiers and zero-padding for integers.